

RiskTool

Technical Documentation & User Manual

Portfolio Risk Optimization & Signal Strategy Framework

Modules:

Global Asset Allocation (GAA)
Signal-Driven Equity/Cash Strategy
Portfolio Risk Optimizer
Volatility Surface Tooling

MATLAB R2024b+

Version 2.0 — February 2026

Confidential — For Internal Use Only

Contents

1	System Overview	2
1.1	Introduction	2
1.2	Architecture	2
1.3	Requirements	2
1.4	Quick Start	2
2	Portfolio Risk Optimizer	4
2.1	Overview	4
2.2	Launch	4
2.3	Tabs and Workflow	4
2.4	Optimization Methods	5
3	Global Asset Allocation (GAA)	6
3.1	Overview	6
3.2	Asset Universe	6
3.3	Nine-Step Pipeline	6
3.4	Capital Market Assumptions (CMA)	7
3.5	SAA Optimization Methods	7
3.6	Tactical Overlay	7
3.7	Covariance Estimation	7
3.8	Risk Decomposition	8
3.9	Command-Line Usage	8
4	Signal Strategy	9
4.1	Overview	9
4.2	Launch	9
4.3	Data Pipeline	9
4.4	Signal Types	9
4.4.1	Momentum	9
4.4.2	EWMAC (Exponentially Weighted Moving Average Crossover)	10
4.4.3	Ensemble (Multi-Lookback Momentum)	10
4.4.4	MacroZ	10
4.4.5	MeanReversion	10
4.4.6	Composite	10
4.5	EWMA Volatility Estimator	11
4.6	Volatility Floor	11
4.7	Activation Functions	11
4.8	Extended Signal Set (12 Signals)	11
4.9	Strategy Mode 1: Composite Signal	12
4.10	Strategy Mode 2: Risk Budget	12
4.11	Strategy Mode 3: Decision Tree	12
4.12	Signal Normalization	12
4.13	Signal-to-Weight Mapping	12
4.14	Backtesting	13
4.15	Stress Testing	13
5	Signal Builder Dialog	14
5.1	Overview	14
5.2	Workflow	14
5.3	Signal Type Panels	14

6	Volatility Surface Utility	15
6.1	Overview	15
6.2	Usage	15
6.3	SVI Model	15
7	GAA Function Reference	16
8	Signal Strategy Function Reference	18
9	Data Structures Reference	19
9.1	Market Data (<code>mkt</code> struct)	19
9.2	Macro Data (<code>macro</code> table)	19
9.3	CMA Struct	19
9.4	Performance Struct	19
10	Testing	20
10.1	Test Suites	20
10.1.1	GAA End-to-End Test (<code>testGAA_synthetic.m</code>)	20
10.1.2	Momentum Upgrades Test (<code>test_momentum_upgrades.m</code>)	20
10.2	Running Tests	20
11	Caching and Data Management	21
11.1	Data Caching	21
11.2	Output Directory	21
12	Academic References	22

1 System Overview

1.1 Introduction

RiskTool is a comprehensive MATLAB framework for quantitative portfolio management. It provides three integrated modules:

1. **Portfolio Risk Optimizer** — Full-featured GUI for risk analysis, optimization, and scenario testing across multiple asset classes.
2. **Global Asset Allocation (GAA)** — Strategic and tactical allocation across 17 asset classes with walk-forward backtesting.
3. **Signal Strategy** — Equity/cash allocation driven by momentum, macro, and composite signals with three strategy modes.

A standalone **Volatility Surface** utility for SPY options completes the toolkit.

1.2 Architecture

The codebase is organized into MATLAB packages with clear separation of concerns:

Layer	Location	Purpose
GUI	Root	App Designer GUIs (<code>PortfolioRiskOptimizerApp</code> , <code>SignalStrategyApp</code> , <code>SignalBuilderDialog</code>)
Entry Points	Root	Command-line launchers (<code>runGAA</code> , <code>run_*App</code>)
GAA Engine	+gaa/	17 functions: config, data, CMA, SAA, tactical, backtest, reporting
Signal Engine	+sigstrat/	16 functions: signals, normalization, mapping, 3 strategy modes
Utilities	Root	Vol surface (<code>GetVolSurface</code>), test suites

1.3 Requirements

- MATLAB R2024b or later (App Designer, `uifigure`, `uigridlayout`)
- **Optimization Toolbox** — required for `fmincon`, `quadprog`
- **Statistics and Machine Learning Toolbox** — required for decision trees, cross-validation, `normcdf`
- **Global Optimization Toolbox** — optional, for genetic algorithm in robust optimization
- Internet access for Stooq and FRED data downloads (cached for 24 hours)

1.4 Quick Start

```

1 %% Launch the Portfolio Risk Optimizer GUI
2 run_PortfolioRiskOptimizerApp
3
4 %% Launch the Signal Strategy GUI
5 run_SignalStrategyApp
6
7 %% Run GAA from the command line
8 results = runGAA('StartDate','20120101', 'Source','Stooq', 'Plot',true)
;
```

```
9
10 %% Query implied volatility
11 iv = GetVolSurface(0.25, 0.95, struct('Type','moneyness','Plot',true));
```

2 Portfolio Risk Optimizer

2.1 Overview

The `PortfolioRiskOptimizerApp` is the main GUI for multi-asset portfolio analysis. It provides:

- Risk metrics: VaR and CVaR via Historical, Parametric, and Monte Carlo methods
- Five optimization objectives: Min Variance, Max Sharpe, Min CVaR, Min VaR, Risk Parity (ERC)
- Strategic Asset Allocation (SAA) and Tactical Asset Allocation (TAA) with combined weights
- Sensitivity analysis (+1% asset shocks) and scenario PnL approximation
- Conditional historical bootstrap via user-defined logical expressions
- Tracking error analysis and attribution against a benchmark
- Performance comparison tab with multiple portfolio configurations

2.2 Launch

```
1 run_PortfolioRiskOptimizerApp    %% GUI launcher
2 app = PortfolioRiskOptimizerApp; %% programmatic access
```

2.3 Tabs and Workflow

The application is organized into tabs. The typical workflow proceeds left to right:

1. **Data & Weights** — Import or download asset data; define SAA and TAA weights; toggle assets active/inactive.
2. **Risk Metrics** — Compute VaR, CVaR, and contribution analysis at the portfolio level.
3. **Optimization** — Run the selected optimizer subject to constraints (long-only, equity/bond bounds, leverage).
4. **Scenario Analysis** — Define macro scenarios, compute approximate PnL impact, and stress-test the portfolio.
5. **Sensitivity** — Per-asset +1% shock impact on portfolio risk and return.
6. **Bootstrap** — Conditional resampling (e.g., “VIX > 25 & SP500_ret < 0”) for tail-risk analysis.
7. **Performance** — Compare strategy vs. benchmark cumulative returns, drawdowns, and rolling metrics.

2.4 Optimization Methods

Method	Description
Min Variance	Minimize portfolio variance $w^\top \Sigma w$ subject to constraints
Max Sharpe	Maximize $(\mu^\top w - r_f) / \sqrt{w^\top \Sigma w}$
Min CVaR	Minimize expected shortfall at the specified confidence level
Min VaR	Minimize Value-at-Risk at the specified confidence level
Risk Parity	Equal Risk Contribution — each asset contributes equally to total portfolio risk

3 Global Asset Allocation (GAA)

3.1 Overview

The GAA module implements a complete strategic and tactical asset allocation framework across 17 asset classes. It can be run from the command line via `runGAA()` or integrated into the Portfolio Risk Optimizer.

3.2 Asset Universe

The default universe (configured in `gaa.gaaConfig`) spans:

Category	Assets	Tickers (Stooq)
Cash & Bonds	Cash, UST Short, UST Long, TIPS, IG Credit, HY Credit	SHV.US, SHY.US, TLT.US, TIP.US, LQD.US, HYG.US
Equities	US Large, US Small, Intl Dev, EM	SPY.US, IWM.US, EFA.US, EEM.US
Alternatives	REITs, Commodities, Gold, Hedge Funds	VNQ.US, DJP.US, GLD.US, QAI.US
FX	DXY (dollar index)	UUP.US

3.3 Nine-Step Pipeline

The `runGAA()` function executes the following pipeline:

1. **Configuration** (`gaa.gaaConfig`) — Load asset universe, tickers, constraints, scenario definitions, and method selections.
2. **Market Data** (`gaa.downloadData`) — Download daily prices from Stooq (or Bloomberg/-Datastream), compute log returns, align dates. Results cached for 24 hours.
3. **Macro Data** (`gaa.downloadFRED`) — Download 12 FRED series and compute 14 derived measures (VIX, yield slope, credit spreads, GDP/CPI growth, etc.).
4. **Capital Market Assumptions** (`gaa.computeCMA`) — Estimate expected returns (μ), volatilities, and the covariance matrix (Σ) via one of three methods.
5. **Strategic Allocation** (`gaa.computeSAA`) — Optimize long-term weights using one of five methods, subject to equity/bond constraints.
6. **Tactical Signals** (`gaa.computeTacticalSignals`) — Compute momentum, risk, and macro signals for tactical overlay.
7. **Combine Allocations** (`gaa.combineAllocations`) — Blend SAA with tactical signals via Black-Litterman or linear overlay.

8. **Walk-Forward Backtest** (`gaa.backtestStrategy`) — Historical simulation with rebalancing, transaction costs, and turnover tracking.
9. **Analysis & Reporting** (`gaa.computePerformance`, `gaa.plotDashboard`, `gaa.generateReport`) — Performance metrics, 6-panel dashboard, and L^AT_EX/PDF report.

3.4 Capital Market Assumptions (CMA)

Method	Description
BuildingBlock	Research Affiliates-style forward estimates combining FRED macro measures with equity risk premia, credit spreads, and real rates.
Historical	Sample mean and covariance from historical returns, with Ledoit–Wolf shrinkage.
BLEquilibrium	Reverse-optimize implied returns from market-cap weights using the Black–Litterman framework.

3.5 SAA Optimization Methods

Method	Description
MinimaxRegret	Robust allocation across 4 growth/inflation scenarios; minimizes worst-case regret.
ScenarioWeighted	Scenario probability-weighted optimal portfolios.
MaxSharpe	Standard mean–variance optimization for maximum Sharpe ratio.
MinVariance	Global minimum variance portfolio.
RiskParity	Equal Risk Contribution (ERC) — each asset contributes equally to total risk.

3.6 Tactical Overlay

Tactical signals are computed from three categories:

- **Momentum** — Time-series (vol-scaled cumulative return) and cross-sectional (demeaned) momentum per asset.
- **Risk** — VIX level, credit spread regime, and volatility regime indicators.
- **Macro** — Yield curve slope, growth, and inflation momentum.

Signals are combined into a composite $\in [-1, +1]$ per asset. Two blending modes are available:

- **Black–Litterman** — Convert signals to views and blend with SAA via the BL posterior.
- **Linear** — Scale signals by the tactical budget and apply as additive deltas to SAA weights.

3.7 Covariance Estimation

Robust covariance estimation is provided by `gaa.robustCovariance`:

- **Ledoit–Wolf Shrinkage** — Shrinks sample covariance toward a scaled identity target, with analytically optimal intensity $\delta \in [0, 1]$.
- **Marcenko–Pastur Denoising** — Optional eigenvalue clipping that replaces noise eigenvalues with their average, improving out-of-sample stability.

3.8 Risk Decomposition

The function `gaa.computeRiskDecomp` provides:

- **MCR** — Marginal Contribution to Risk: $\partial\sigma_p/\partial w_i$.
- **CR** — Contribution to Risk: $w_i \times \text{MCR}_i$.
- **PCT** — Percentage risk contribution: CR_i/σ_p .
- **HHI** — Herfindahl–Hirschman Index of risk concentration.
- **ENB** — Effective Number of Bets: $1/\text{HHI}$.
- **DR** — Diversification Ratio: $w^\top \sigma/\sigma_p$.

3.9 Command-Line Usage

```

1 %% Full pipeline with defaults
2 results = runGAA();
3
4 %% Custom configuration
5 results = runGAA('StartDate', '20150101', ...
6   'Source',      'Stooq', ...
7   'CMAMethod',  'BuildingBlock', ...
8   'SAAMethod',  'MinimaxRegret', ...
9   'CombineMethod', 'BL', ...
10  'TacticalBudget', 0.20, ...
11  'Plot', true);

```

4 Signal Strategy

4.1 Overview

The Signal Strategy module implements an equity/cash allocation system driven by quantitative signals. It supports three distinct strategy modes, each with its own optimization approach.

4.2 Launch

```
1 run_SignalStrategyApp    %% GUI
```

4.3 Data Pipeline

1. **Market Data** (`sigstrat.downloadMarketData`) — S&P 500 and T-Bill daily returns from Stooq, with SPY.US fallback and 24h caching.
2. **Macro Data** (`sigstrat.downloadMacroData`) — 12 FRED series with 14 derived measures; shares pipeline with GAA where available.
3. **Signal Generation** — Three sources:
 - Demo signals (`sigstrat.generateDemoSignals`) — 5 quick signals for exploration
 - Extended signals (`sigstrat.generateExtendedSignals`) — 12 production signals across fast/medium/slow categories
 - Custom signals (`SignalBuilderDialog`) — Interactive builder with 6 signal types

4.4 Signal Types

The `sigstrat.buildSignal` function supports six signal types, all producing output bounded in $[-1, +1]$:

4.4.1 Momentum

Vol-adjusted price momentum with expanding z-score and configurable activation function.

Parameter	Default	Range	Description
<code>lookback</code>	126	21–504	Lookback window in trading days
<code>minHistory</code>	252	42–1008	Minimum observations for expanding z-score
<code>volHalflife</code>	32	10–120	EWMA volatility halflife (days)
<code>skipDays</code>	0	0–21	Skip recent days to avoid short-term reversal
<code>volFloor</code>	0	≥ 0	Min vol estimate; 0 = auto (5th percentile)
<code>activation</code>	<code>tanh</code>	—	Activation function (see below)
<code>tanhScale</code>	2	0.1–10	Scaling divisor for activation

The momentum signal at time t is:

$$\text{mom}_t = \frac{\sum_{i=t-L-S+1}^{t-S} r_i}{\hat{\sigma}_t^{\text{EWMA}} \cdot \sqrt{L}} \quad (1)$$

where L is `lookback`, S is `skipDays`, and $\hat{\sigma}_t^{\text{EWMA}}$ is the EWMA volatility.

4.4.2 EWMAC (Exponentially Weighted Moving Average Crossover)

EMA fast/slow crossover trend signal, following Levine–Pedersen (2016). Equivalent to TSMOM but smoother with lower turnover.

Parameter	Default	Range	Description
fastSpan	16	2–128	Fast EMA span
slowSpan	64	8–512	Slow EMA span
volHalflife	32	10–120	EWMA vol halflife
volFloor	0	≥ 0	Min vol estimate
activation	tanh	—	Activation function
tanhScale	2	0.1–10	Activation scale

The raw EWMAC signal is:

$$\text{EWMAC}_t = \frac{\text{EMA}_{\text{fast}}(P_t) - \text{EMA}_{\text{slow}}(P_t)}{\hat{\sigma}_t^{\text{EWMA}} \cdot P_t} \quad (2)$$

4.4.3 Ensemble (Multi-Lookback Momentum)

Combines momentum signals across multiple lookback windows. This is the single most impactful enhancement per practitioners (Moskowitz–Ooi–Pedersen 2012, Baltas–Kosowski 2020).

Parameter	Default	Range	Description
lookbacks	[21 63 126 252]	—	Vector of lookback windows
method	equal	—	equal (equal weight) or riskparity ($1/\sigma$ weighting)
volHalflife	32	10–120	EWMA vol halflife
skipDays	0	0–21	Skip recent days
volFloor	0	≥ 0	Min vol estimate
activation	tanh	—	Activation function

4.4.4 MacroZ

Expanding robust z-score of a macro variable (e.g., VIX, HY_Spread). Uses median/MAD for robustness to outliers.

Parameter	Default	Range	Description
variable	VIX	—	Macro column name
minWindow	126	20–504	Min observations for z-score
negate	false	—	Flip sign (e.g., lower VIX = bullish)
tanhScale	2	0.1–10	Scaling divisor

4.4.5 MeanReversion

Distance from rolling moving average, z-scored and sign-flipped. Price above MA $\rightarrow$ expect reversion down $\rightarrow$ bearish signal.

4.4.6 Composite

Weighted average of multiple macro z-scores. User defines the variables, weights, and per-variable sign flips.

4.5 EWMA Volatility Estimator

All momentum-family signals (Momentum, EWMA, Ensemble) use EWMA volatility instead of equal-weight standard deviation:

$$\hat{\sigma}_t^2 = \lambda \hat{\sigma}_{t-1}^2 + (1 - \lambda) r_t^2, \quad \lambda = \exp\left(-\frac{\ln 2}{h}\right) \quad (3)$$

where h is the halflife in days (default 32). This places more weight on recent observations, producing a smoother and more responsive vol estimate.

4.6 Volatility Floor

To prevent extreme leverage in low-volatility regimes, a vol floor is applied:

$$\hat{\sigma}_t = \max(\hat{\sigma}_t^{\text{EWMA}}, \text{floor}) \quad (4)$$

When `volFloor` = 0 (auto), the floor is the 5th percentile of the trailing 504-day EWMA vol series. When > 0 , it is used directly.

4.7 Activation Functions

Three activation functions map the z-scored signal to $[-1, +1]$:

Name	Formula	Behavior
<code>tanh</code>	$\tanh(z/s)$	Standard S-curve, saturates at ± 1 (default)
<code>sigmoid</code>	$2\Phi(z/s) - 1$	Normal CDF scaled to $[-1, +1]$; lighter tails than tanh
<code>revertSigmoid</code>	$c \cdot (z/s) \cdot e^{-z^2/(2s^2)}$	Dao et al. (2021): position rises then falls for extreme signals, providing crash protection

where $s = \text{tanhScale}$, Φ = standard normal CDF, and $c = (1 + 2/s^2)^{3/4}$.

4.8 Extended Signal Set (12 Signals)

The function `sigstrat.generateExtendedSignals` produces 12 production signals:

#	Name	Speed	Description
1	SPX_Mom_1m	Fast	21d equity momentum
2	SPX_Mom_3m	Fast	63d equity momentum
3	VIX_Fast	Fast	Negated 63d VIX z-score
4	CreditFast	Fast	Negated 63d HY spread z-score
5	SPX_Mom_6m	Medium	126d equity momentum
6	VIX_Med	Medium	Negated 126d VIX z-score
7	YieldSlope_Med	Medium	126d yield slope z-score
8	FinConditions	Medium	VIX + credit + yield composite
9	SPX_Mom_12m	Slow	252d equity momentum
10	VIX_Slow	Slow	Negated 252d VIX z-score
11	MacroTrend	Slow	GDP + retail + industrial composite
12	CreditSlow	Slow	Negated 252d HY spread z-score

4.9 Strategy Mode 1: Composite Signal

The `sigstrat.optimizeComposite` function:

1. Computes a weighted average of normalized signals: $C_t = \sum_i w_i \cdot s_{i,t}$
2. Maps C_t to equity weight via step-function thresholds
3. Optimizes weights and thresholds via grid search (in-sample Sharpe + return + drawdown penalty)
4. Reports in-sample and out-of-sample performance

4.10 Strategy Mode 2: Risk Budget

The `sigstrat.riskBudgetStrategy` function:

1. Allocates a risk budget to each signal (default: equal)
2. Maps each signal to equity weight via a configurable translation function (Linear, Threshold, Sigmoid, Quantile, Piecewise, Spline, or Power)
3. Combines per-signal allocations weighted by budget
4. Supports optional macro conditioning

4.11 Strategy Mode 3: Decision Tree

The `sigstrat.decisionTreeStrategy` function:

1. **Rule-Based** — User-defined rules (variable, operator, threshold $\rightarrow$ equity weight)
2. **Auto-Optimize** — CART regression tree auto-fitted to predict equity weight $\in [0, 1]$
3. Walk-forward cross-validation with expanding windows
4. Variable importance metrics from the fitted tree

4.12 Signal Normalization

The `sigstrat.normalizeSignals` function supports 5 methods:

Method	Description
RobustZ	Expanding robust z-score (median/MAD) $\rightarrow$ tanh (default)
StandardZ	Expanding standard z-score $\rightarrow$ tanh
RollingZ	Rolling-window z-score $\rightarrow$ tanh
Percentile	Expanding percentile rank mapped to $[-1, +1]$
MinMax	Expanding min-max scaling to $[-1, +1]$

4.13 Signal-to-Weight Mapping

The `sigstrat.mapSignalToWeight` function maps signals from $[-1, +1]$ to equity weights $[0, 1]$:

Method	Description
Step	Threshold-based discrete levels
Linear	$(\text{signal} + 1)/2$
Sigmoid	Smooth sigmoid mapping
PiecewiseLinear	User-defined breakpoints
Spline	Smooth spline interpolation
Power	Power law transformation

4.14 Backtesting

The `sigstrat.backtestEquityCash` function performs walk-forward simulation:

- Configurable rebalancing frequency (daily, weekly, monthly)
- Transaction cost modeling (basis points per trade)
- Weight drift tracking between rebalance dates
- Benchmark comparisons (60/40, 100% equity)

4.15 Stress Testing

The `sigstrat.stressTestStrategy` function evaluates performance during 6 historical crisis episodes:

Episode	Period	Event
GFC	Oct 2007 – Mar 2009	Global Financial Crisis
Euro Crisis	Jul 2011 – Oct 2011	European sovereign debt
Taper Tantrum	May 2013 – Sep 2013	Fed taper announcement
2018 Q4	Oct 2018 – Dec 2018	Rate hike selloff
COVID	Feb 2020 – Mar 2020	Pandemic crash
Rate Shock	Jan 2022 – Oct 2022	Inflation and rate hiking cycle

5 Signal Builder Dialog

5.1 Overview

The `SignalBuilderDialog` is a modal UI launched from the Signal Strategy App for building individual custom signals interactively.

5.2 Workflow

1. Select signal type from the dropdown (6 types available)
2. Configure parameters in the dynamic panel (parameters change per type)
3. Enter a unique signal name
4. Click **Preview** to visualize the signal and inspect statistics
5. Click **Add Signal** to accept, or **Cancel** to discard

5.3 Signal Type Panels

Each signal type presents a tailored parameter panel:

- **Momentum** — Lookback, Min History, Vol Halflife, Skip Days, Vol Floor, Activation, Tanh Scale
- **EWMAC** — Fast Span, Slow Span, Vol Halflife, Vol Floor, Activation, Tanh Scale
- **Ensemble** — Lookbacks (comma-separated text), Method (Equal Weight / Risk Parity), Vol Halflife, Skip Days, Vol Floor, Activation, Tanh Scale
- **MacroZ** — Variable dropdown, Min Window, Negate checkbox, Tanh Scale
- **MeanReversion** — MA Lookback, Min History, Tanh Scale
- **Composite** — Editable table (Variable, Weight, Negate per row), Min Window, Tanh Scale

6 Volatility Surface Utility

6.1 Overview

The `GetVolSurface` function downloads SPY options, calibrates an SVI (Stochastic Volatility Inspired) surface, and queries implied volatility at any (T, K) point.

6.2 Usage

```

1 %% Query IV at 3-month, 95% moneyness
2 iv = GetVolSurface(0.25, 0.95, struct('Type','moneyness'));
3
4 %% Query IV at 1-year, strike = 500
5 iv = GetVolSurface(1.0, 500, struct('Type','strike'));
6
7 %% Force refresh and plot 3D surface
8 iv = GetVolSurface(0.5, 1.0, struct('Type','moneyness', ...
9   'Refresh',true, 'Plot',true));

```

6.3 SVI Model

The SVI parameterization models total implied variance as a function of log-moneyness k :

$$w(k) = a + b \left(\rho(k - m) + \sqrt{(k - m)^2 + \sigma^2} \right) \quad (5)$$

Parameters (a, b, ρ, m, σ) are calibrated per expiry via least-squares fit to market quotes. The function uses persistent caching (`VolSurfaceCache.mat`) and a 24-hour refresh policy.

7 GAA Function Reference

Function	Description
<code>gaaConfig</code>	Central configuration: asset universe (17 assets), tickers, FRED series, scenario definitions, optimization constraints, and method selections. Returns a struct used by all downstream functions. <code>cfg = gaa.gaaConfig()</code>
<code>downloadData</code>	Download daily prices from Stooq/Bloomberg/Datastream. Computes log returns, aligns dates across assets. Caches results for 24 hours. <code>[R, prices, dates, names] = gaa.downloadData(cfg)</code>
<code>downloadFRED</code>	Download 12 FRED series and compute 14 derived macro measures (VIX, yield slope, credit spreads, GDP/CPI growth, DXY, NFP, etc.). <code>macro = gaa.downloadFRED(cfg)</code>
<code>computeCMA</code>	Estimate Capital Market Assumptions: expected returns, volatilities, and covariance matrix via BuildingBlock, Historical, or BLE-equilibrium methods. <code>cma = gaa.computeCMA(R, cfg, macro)</code>
<code>computeSAA</code>	Compute Strategic Asset Allocation via MinimaxRegret, ScenarioWeighted, MaxSharpe, MinVariance, or RiskParity. <code>saa = gaa.computeSAA(cma, cfg)</code>
<code>computeTacticalSignals</code>	Compute tactical signals from momentum, risk, and macro categories. Returns composite signal per asset in [-1,+1]. <code>signals = gaa.computeTacticalSignals(R, macro, cfg)</code>
<code>combineAllocations</code>	Blend SAA with tactical signals via Black-Litterman or linear overlay. Enforces tactical budget and per-asset deviation constraints. <code>alloc = gaa.combineAllocations(saa, signals, cma, cfg)</code>
<code>backtestStrategy</code>	Walk-forward historical backtest with rebalancing, transaction costs, turnover tracking, and SAA benchmark. <code>bt = gaa.backtestStrategy(alloc, R, cfg)</code>
<code>computePerformance</code>	Comprehensive performance analytics: annualized metrics, draw-down, rolling statistics, monthly returns, attribution. <code>perf = gaa.computePerformance(bt)</code>
<code>computeRiskDecomp</code>	Risk decomposition: MCR, CR, percentage contributions, HHI, ENB, and Diversification Ratio. <code>rd = gaa.computeRiskDecomp(w, Sigma)</code>
<code>optimizePortfolio</code>	Unified optimizer: MaxSharpe, MinVariance, RiskParity, MinCVaR, TargetVol, Resampled. Long-only and constrained. <code>w = gaa.optimizePortfolio(mu, Sigma, opts)</code>
<code>blackLitterman</code>	Black-Litterman posterior returns and covariance from views. <code>[muBL, SigBL] = gaa.blackLitterman(Sigma, wMkt, P, Q, opts)</code>
<code>robustCovariance</code>	Ledoit-Wolf shrinkage with optional Marcenko-Pastur eigenvalue denoising.

Function	Description
	<code>[S, info] = gaa.robustCovariance(R, opts)</code>
<code>ledoitWolfShrink</code>	Ledoit-Wolf (2004) shrinkage toward scaled identity. <code>[S, delta] = gaa.ledoitWolfShrink(R)</code>
<code>plotDashboard</code>	6-panel dashboard: CMA bars, signal heatmap, allocation area, risk waterfall, cumulative performance, drawdown. <code>fig = gaa.plotDashboard(results)</code>
<code>generateReport</code>	LaTeX/PDF report generation with embedded dashboard figure. <code>gaa.generateReport(results)</code>

8 Signal Strategy Function Reference

Function	Description
<code>downloadMarketData</code>	Download S P 500 and T-Bill daily data from Stooq. Returns struct with dates, returns, and prices. 24h cache. <code>mkt = sigstrat.downloadMarketData()</code>
<code>downloadMacroData</code>	Download FRED macro series with 14 derived measures. Falls back to embedded pipeline if <code>gaa.downloadFRED</code> unavailable. <code>macro = sigstrat.downloadMacroData(dates)</code>
<code>generateDemoSignals</code>	Generate 5 demo signals: yield slope, VIX, credit spread, momentum, macro composite. <code>[data, names] = sigstrat.generateDemoSignals(macro, mkt)</code>
<code>generateExtendedSignals</code>	Generate 12 production signals across fast/medium/slow categories. Uses EWMA vol. <code>[data, names] = sigstrat.generateExtendedSignals(macro, mkt)</code>
<code>buildSignal</code>	Build custom signal: Momentum, EWMAC, Ensemble, MacroZ, MeanReversion, or Composite. <code>[sig, meta] = sigstrat.buildSignal(type, params, mkt, macro)</code>
<code>normalizeSignals</code>	Normalize signals to [-1,+1] via RobustZ, StandardZ, RollingZ, Percentile, or MinMax. <code>normed = sigstrat.normalizeSignals(data, method)</code>
<code>mapSignalToWeight</code>	Map signal [-1,+1] to equity weight [0,1] via Step, Linear, Sigmoid, Piecewise, Spline, or Power. <code>w = sigstrat.mapSignalToWeight(sig, method, opts)</code>
<code>backtestEquityCash</code>	Walk-forward equity/cash backtest with rebalancing, costs, and benchmark comparisons. <code>bt = sigstrat.backtestEquityCash(w, mkt, opts)</code>
<code>computeStrategyPerf</code>	Performance metrics for strategy, 60/40, and 100 <code>perf = sigstrat.computeStrategyPerf(bt)</code>
<code>stressTestStrategy</code>	Evaluate strategy during 6 historical crisis episodes. <code>st = sigstrat.stressTestStrategy(bt)</code>
<code>optimizeComposite</code>	Mode 1: grid-search optimal weights and thresholds for composite signal strategy. <code>res = sigstrat.optimizeComposite(signals, mkt, opts)</code>
<code>riskBudgetStrategy</code>	Mode 2: per-signal risk budget allocation with configurable translation functions. <code>res = sigstrat.riskBudgetStrategy(signals, mkt, opts)</code>
<code>robustOptimize</code>	K-fold CV robust optimization with L2 regularization and turnover penalty. <code>res = sigstrat.robustOptimize(signals, mkt, opts)</code>
<code>decisionTreeStrategy</code>	Mode 3: rule-based or CART auto-optimized decision tree for weight prediction. <code>res = sigstrat.decisionTreeStrategy(signals, mkt, opts)</code>

9 Data Structures Reference

9.1 Market Data (`mkt struct`)

Field	Type	Description
<code>dates</code>	datetime	Price dates
<code>retDates</code>	datetime	Return dates (<code>dates(2:end)</code>)
<code>spxRet</code>	Tx1 double	Daily S&P 500 log returns
<code>cashRet</code>	Tx1 double	Daily T-Bill returns
<code>spxPrice</code>	Tx1 double	S&P 500 price level

9.2 Macro Data (`macro table`)

A MATLAB table with `Date` column plus derived measures:

- `VIX`
- `FedFunds`
- `TenYr`
- `TwoYr`
- `YieldSlope`
- `HY_Spread`
- `IG_Spread`
- `GDP_YoY`
- `CPI_YoY`
- `CPI_MoM`
- `RetailSales_YoY`
- `IndProd_YoY`
- `NFP_MoM`
- `DEXY_3m`

9.3 CMA Struct

Field	Type	Description
<code>mu</code>	Nx1	Expected annualized returns
<code>vol</code>	Nx1	Expected annualized volatilities
<code>Sigma</code>	NxN	Covariance matrix
<code>Corr</code>	NxN	Correlation matrix
<code>method</code>	char	Method used (BuildingBlock, Historical, BLEquilibrium)

9.4 Performance Struct

Field	Type	Description
<code>annReturn</code>	scalar	Annualized return
<code>annVol</code>	scalar	Annualized volatility
<code>sharpe</code>	scalar	Sharpe ratio
<code>sortino</code>	scalar	Sortino ratio
<code>calmar</code>	scalar	Calmar ratio (return / max drawdown)
<code>maxDD</code>	scalar	Maximum drawdown
<code>hitRate</code>	scalar	Fraction of positive return days
<code>skew</code>	scalar	Return skewness
<code>kurt</code>	scalar	Return kurtosis

10 Testing

10.1 Test Suites

Two test suites validate the framework:

10.1.1 GAA End-to-End Test (`testGAA_synthetic.m`)

Uses synthetic data (10 years, 17 assets, 3-factor correlation structure) to test every module in the GAA pipeline without requiring internet access. Validates:

- Data download and alignment
- CMA computation (all 3 methods)
- SAA optimization (all 5 methods)
- Tactical signal generation
- Allocation blending
- Backtest execution
- Performance computation

10.1.2 Momentum Upgrades Test (`test_momentum_upgrades.m`)

Unit tests for the enhanced momentum signal system. 12 test cases covering:

- Momentum default parameters and valid output range
- EWMA vol produces different signals with different halflife values
- SkipDays produces shifted signals
- All 3 activation functions produce valid $[-1, +1]$ output
- EWMAC with default and custom spans
- Ensemble with equal-weight and risk-parity methods
- Vol floor prevents extreme values
- Extended signals produce $T \times 12$ output
- Backward compatibility of MacroZ

10.2 Running Tests

```
1 %% Run all tests
2 runtests('testGAA_synthetic');
3 runtests('test_momentum_upgrades');
4
5 %% Run with verbose output
6 results = runtests('test_momentum_upgrades', 'Verbosity', 3);
7 disp(results);
```

11 Caching and Data Management

11.1 Data Caching

All data download functions implement a 24-hour disk cache to minimize API calls:

Module	Cache Location	Contents
<code>gaa.downloadData</code>	<code>tempdir/gaa_cache/</code>	Market prices, returns, dates
<code>gaa.downloadFRED</code>	<code>tempdir/gaa_cache/</code>	FRED macro series
<code>sigstrat.downloadMarketData</code>	<code>tempdir/sigstrat_cache/</code>	S&P 500, T-Bill data
<code>GetVolSurface</code>	<code>VolSurfaceCache.mat</code>	SVI parameters per expiry

To force a data refresh, delete the cache directory or pass the **Refresh** option where available.

11.2 Output Directory

The GAA module writes reports and figures to `gaa_output/`:

- `GAA_Report.pdf` — Auto-generated L^AT_EX/PDF strategy report
- Dashboard PNG exports

12 Academic References

The signal construction and portfolio methodology draws on the following research:

- Moskowitz, T., Ooi, Y.H., & Pedersen, L.H. (2012). “Time series momentum.’ *Journal of Financial Economics*, 104(2), 228–250.
- Baltas, N. & Kosowski, R. (2013/2020). “Momentum strategies in futures markets and trend-following funds.’ Working paper / *Journal of Finance and Data Science*.
- Levine, A. & Pedersen, L.H. (2016). “Which trend is your friend?’ *Financial Analysts Journal*, 72(3), 51–66.
- Dao, T.-L., Nguyen, T.-T., Deremble, C., Lemérle, Y., Bouchaud, J.-P., & Potters, M. (2021). “Tail protection for long investors: Trend convexity at work.’ Working paper.
- Novy-Marx, R. (2012). “Is momentum really momentum?’ *Journal of Financial Economics*, 103(3), 429–453.
- Black, F. & Litterman, R. (1992). “Global portfolio optimization.’ *Financial Analysts Journal*, 48(5), 28–43.
- Ledoit, O. & Wolf, M. (2004). “A well-conditioned estimator for large-dimensional covariance matrices.’ *Journal of Multivariate Analysis*, 88(2), 365–411.
- Michaud, R.O. (1998). *Efficient Asset Management*. Harvard Business School Press.
- Carver, R. (2015). *Systematic Trading*. Harriman House.
- Gatheral, J. (2004). “A parsimonious arbitrage-free implied volatility parameterization with application to the valuation of volatility derivatives.’ Presentation, Global Derivatives.